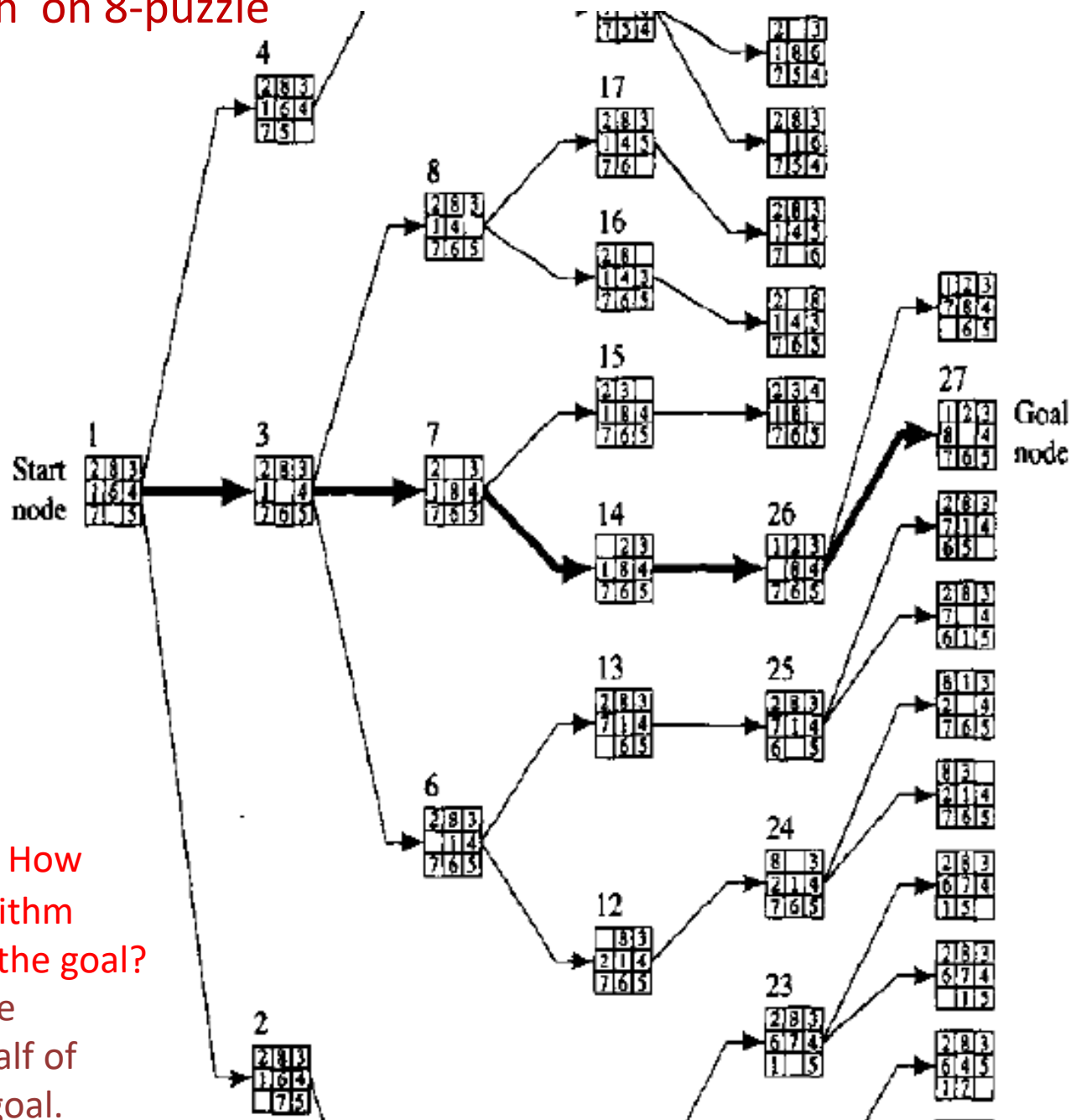


Breadth-First Search on 8-puzzle



Homework-BFS1: How many states the algorithm have searched to get the goal? Assume on level 5, the algorithm searched half of the states to get the goal.

DFS: Recursive calls to DFS-VISIT()

Homework:DFS1

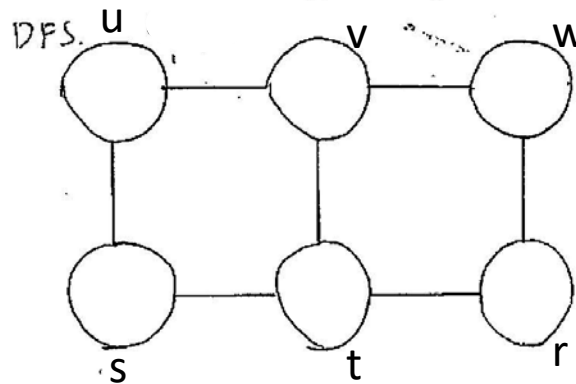
1. Draw the DFS search tree on this graph.

2. Identify backtrack nodes.

Question: Vertex 1 has two adjacent vertices. Who will be visited first?

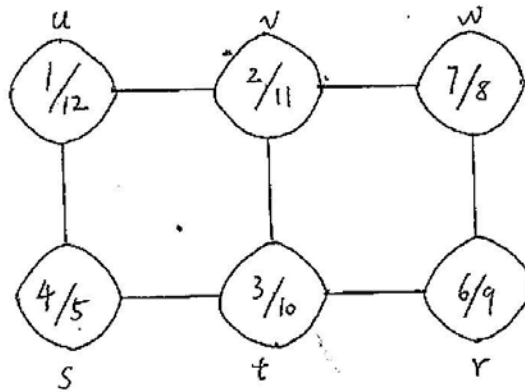
Answer: Depends on the order links in the adjacency list.

DFS: Recursive calls to DFS-VISIT()



Graph Representation
edgelist (undirected)

- (u v)
- (u s)
- (v t)
- (v w)
- (s t)
- (t r)
- (w r)

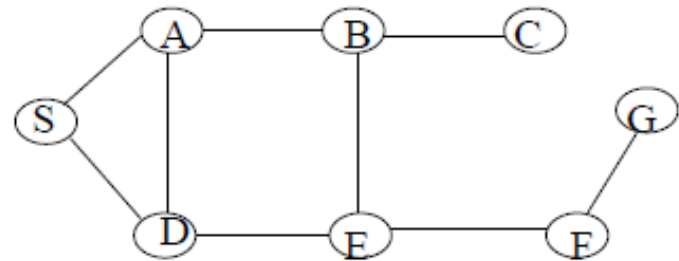
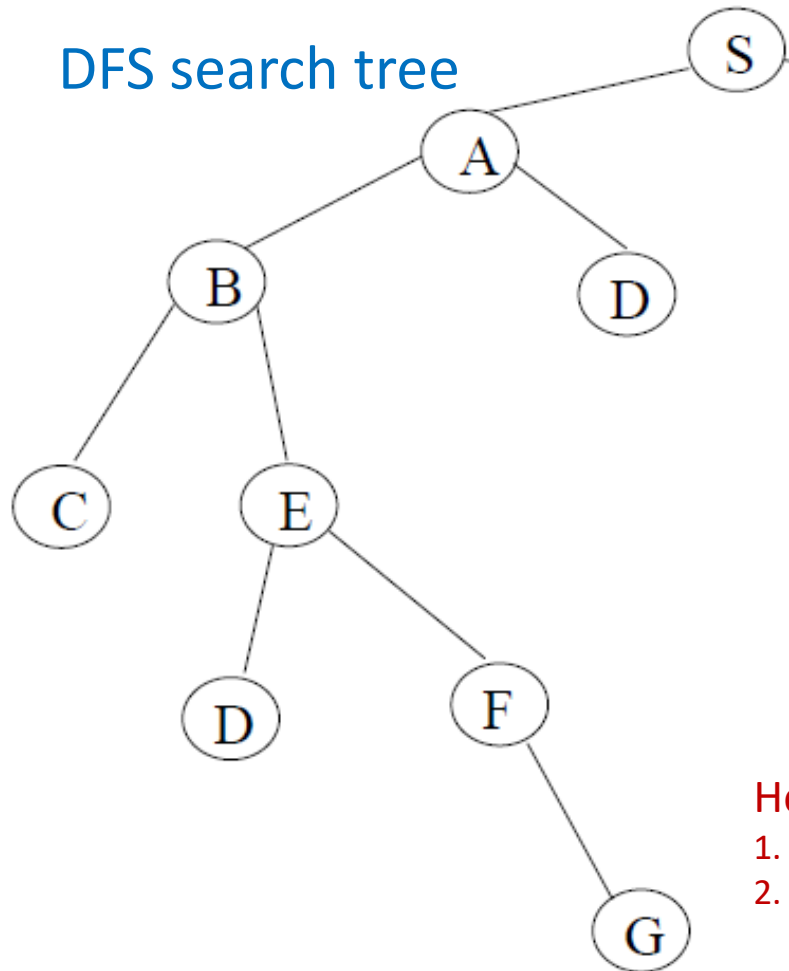


```

1. DFS-VISIT(u)
2.   DFS-VISIT(v)
3.     DFS-VISIT(t)
4.       DFS-VISIT(s)
5.         EXIT(s)
6.       DFS-VISIT(r)
7.         DFS-VISIT(w)
8.           EXIT(w)
9.         EXIT(r)
10.      EXIT(t)
11.    EXIT(v)
12.  EXIT(u)
  
```

Depth-First Search (DFS)

DFS search tree



Homework:DFS2

1. Mark the entering and leave time for every nodes.
2. Mark the backtrack nodes on the tree..

Homework: DFS3

Start (3,3) → Goal (5,4)

P1. DFS backtrack. Find a path from start node to goal node.

Right figure shows a DFS visit route. It hits a dead-end at (3,2).

Your task:

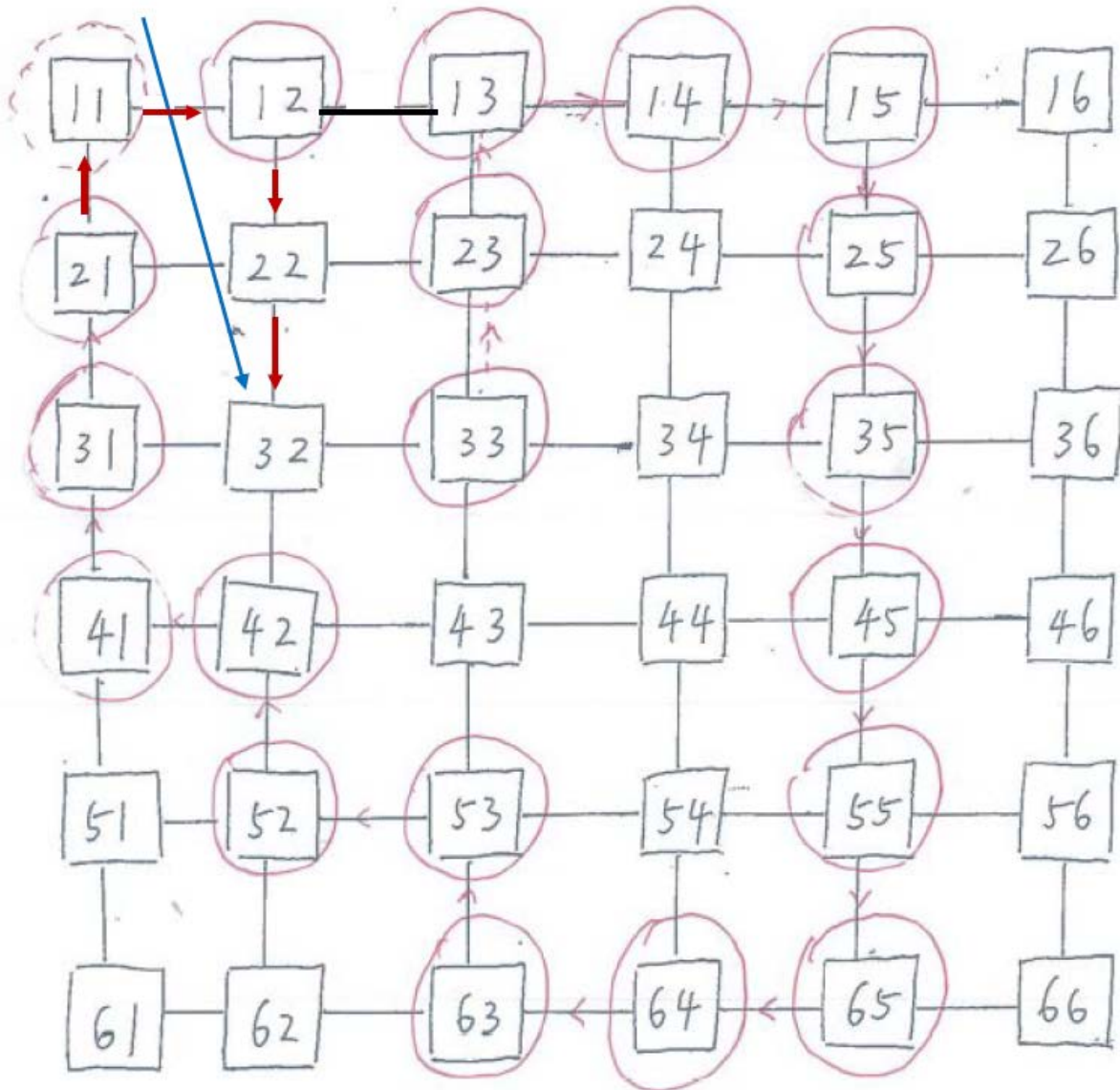
(a) Continue DFS. Do the backtrack and find the first **backtrack node** where DFS can starting visit another route. Mark this node as **bf**.

Mark this node as **bf**.

(b) If your path reach the goal, stop. Otherwise do backtrack and find the 2nd backtrack node, mark it. Follow step (a).

Repeat, until you find a path that hits the goal.

DFS hits a dead-end

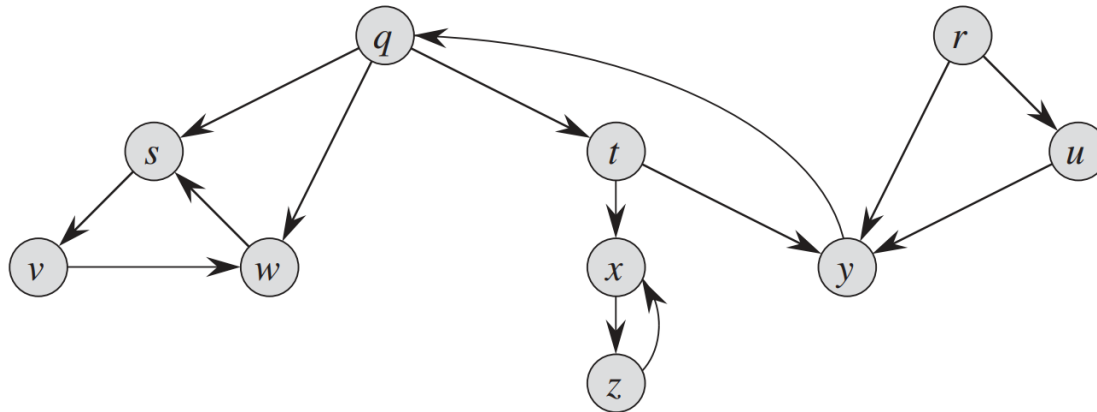


Homework: DFS4

Show how DFS visit all nodes in the following graph. Starting from node v

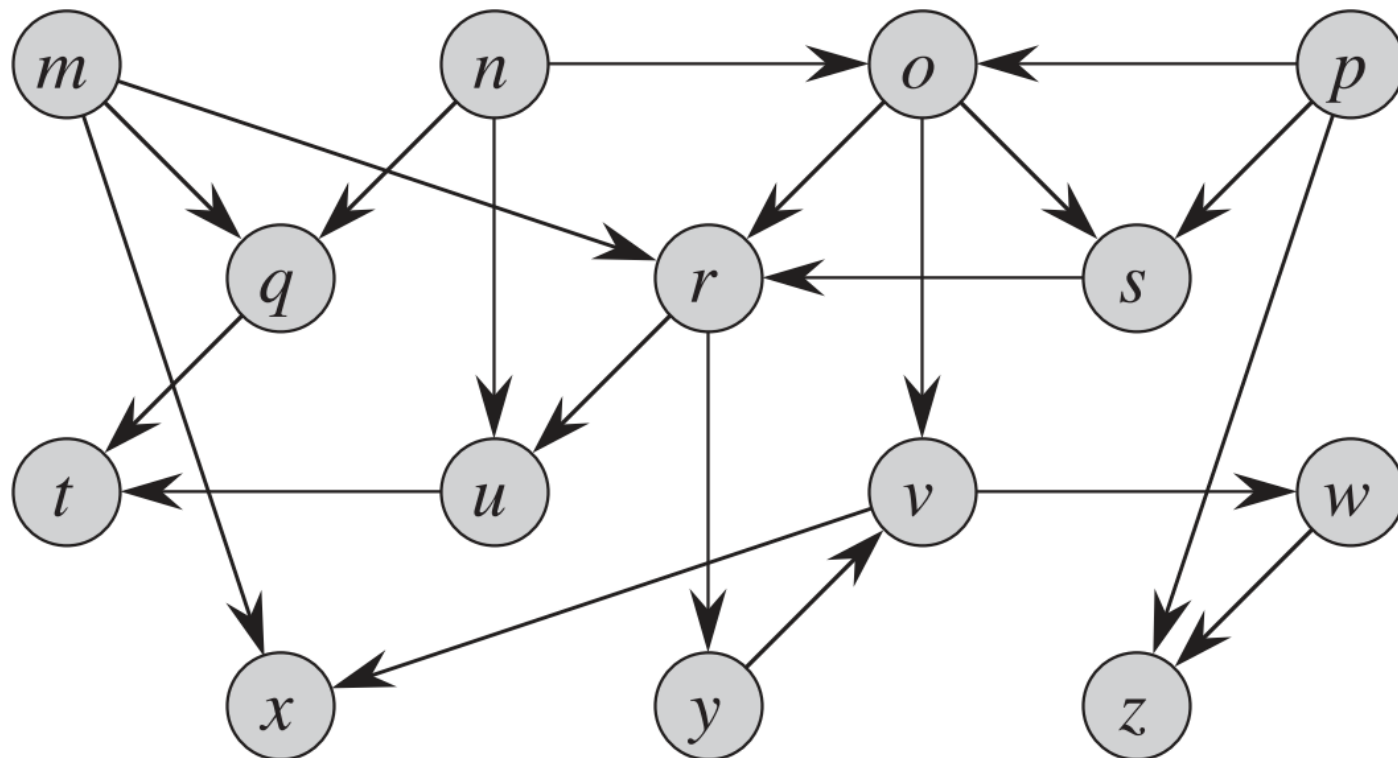
Assume: In DFS(), the nodes are visited alphabetically.

Assume: The adjacency nodes of each nodes are listed alphabetically.

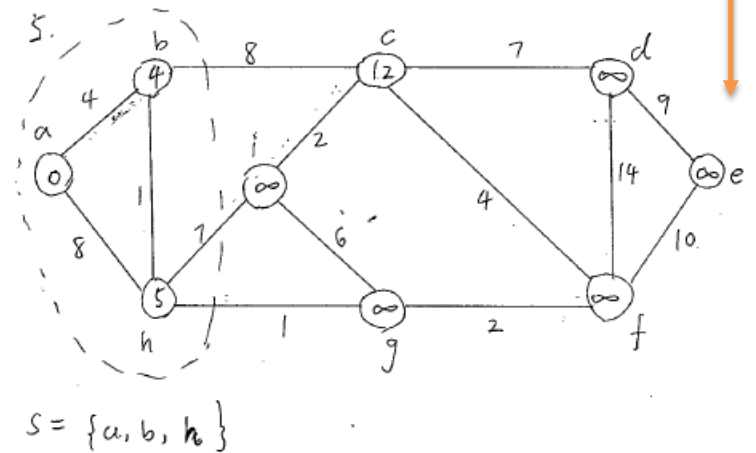
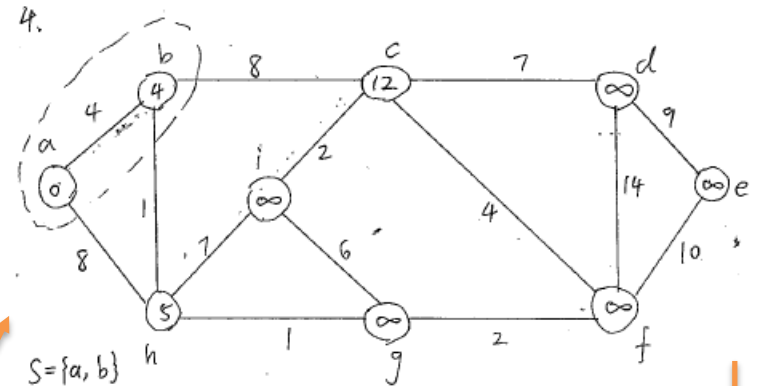
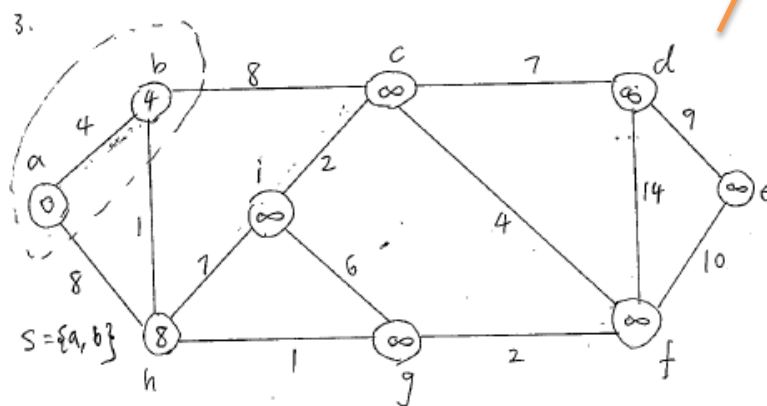
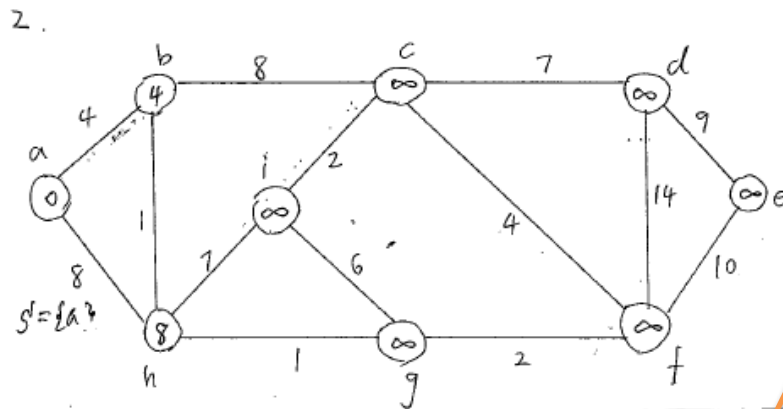
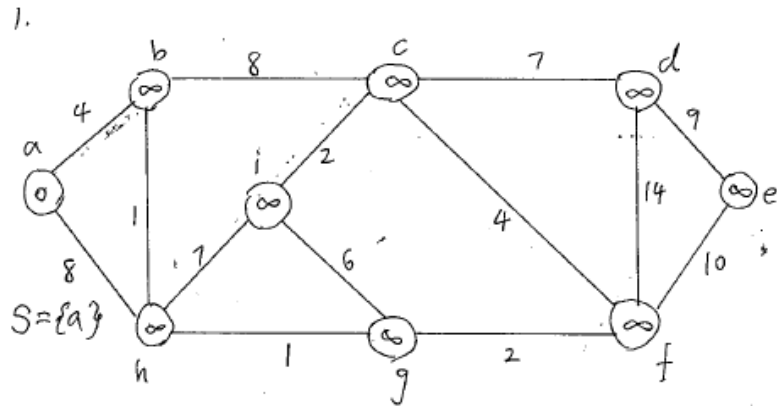


Homework: DFS5:

Finding the topological sort/ordering of nodes in following graph

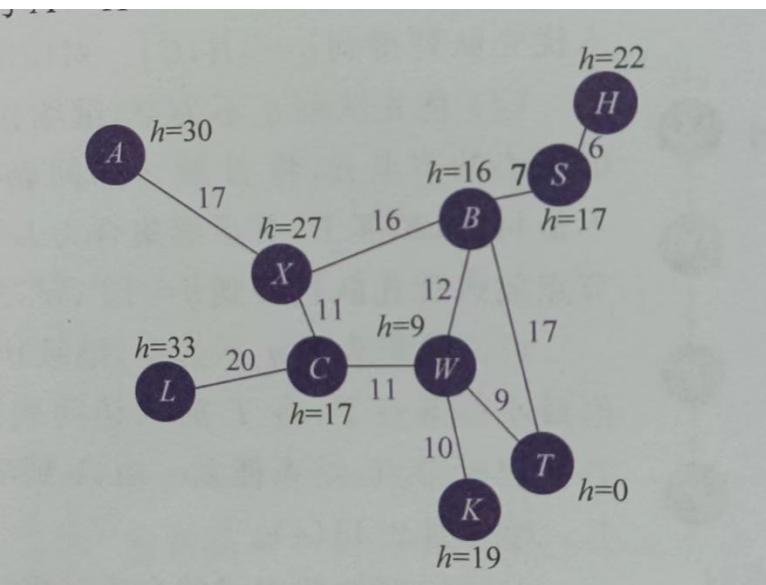


Another Example of Dijkstra Algorithm



HOMEWORK: Dijkstra1.
Complete the algorithm on this graph.

Homework A star 1: Running A* on the graph. A is start, T is goal. Your task: add $g(n)$ and $f(n)$ computation in each iteration of the while loop. See A* algorithm details.



初始化时, 优先队列为空 $q = \{\}$, 访问表为空 $v = \{\}$ 。算法执行过程如下:

(1) 将初始节点 A 放入优先队列得到 $q = \{A\}$ 。

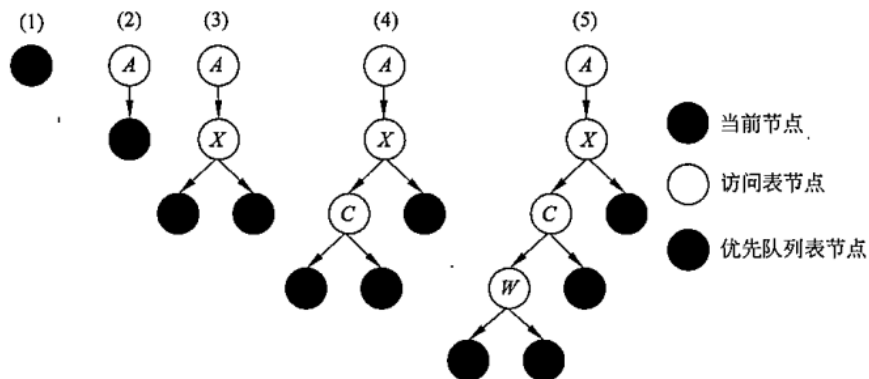
(2) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 A, 将 A 放入访问表得到 $v = \{A\}$ 。A 不是目标, 扩展 A, 其后继集合为 $\{X\}$ 。将后继集合中的节点放入优先队列得到 $q = \{X\}$ 。对应图 2.12(1)。

(3) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 X, 将 X 放入访问表得到 $v = \{A, X\}$ 。X 不是目标, 扩展 X, 其后继集合为 $\{B, C\}$ 。将后继集合中的节点放入优先队列得到 $q = \{B, C\}$ 。对应图 2.12(2)。

(4) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 C, 将 C 放入访问表得到 $v = \{A, X, C\}$ 。C 不是目标, 扩展 C, 其后继集合为 $\{L, W\}$ 。将后继集合中的节点放入优先队列得到 $q = \{W, B, L\}$ 。对应图 2.12(3)。

(5) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 W, 将 W 放入访问表 $v = \{A, X, C, W\}$ 。W 不是目标, 扩展 W, 其后继集合为 $\{T, K\}$ 。将后继集合中的节点放入优先队列得到 $q = \{T, B, K, L\}$ 。对应图 2.12(4)。

(6) 优先队列 q 不为空, 继续执行。弹出优先队列中 f 值最小的节点 T, 将 T 放入访问表得到 $v = \{A, X, C, W, T\}$ 。T 为目标节点, 结束搜索。由 A 到 T 的路径为 $A \rightarrow X \rightarrow C \rightarrow W \rightarrow T$ 。对应图 2.12(5)。

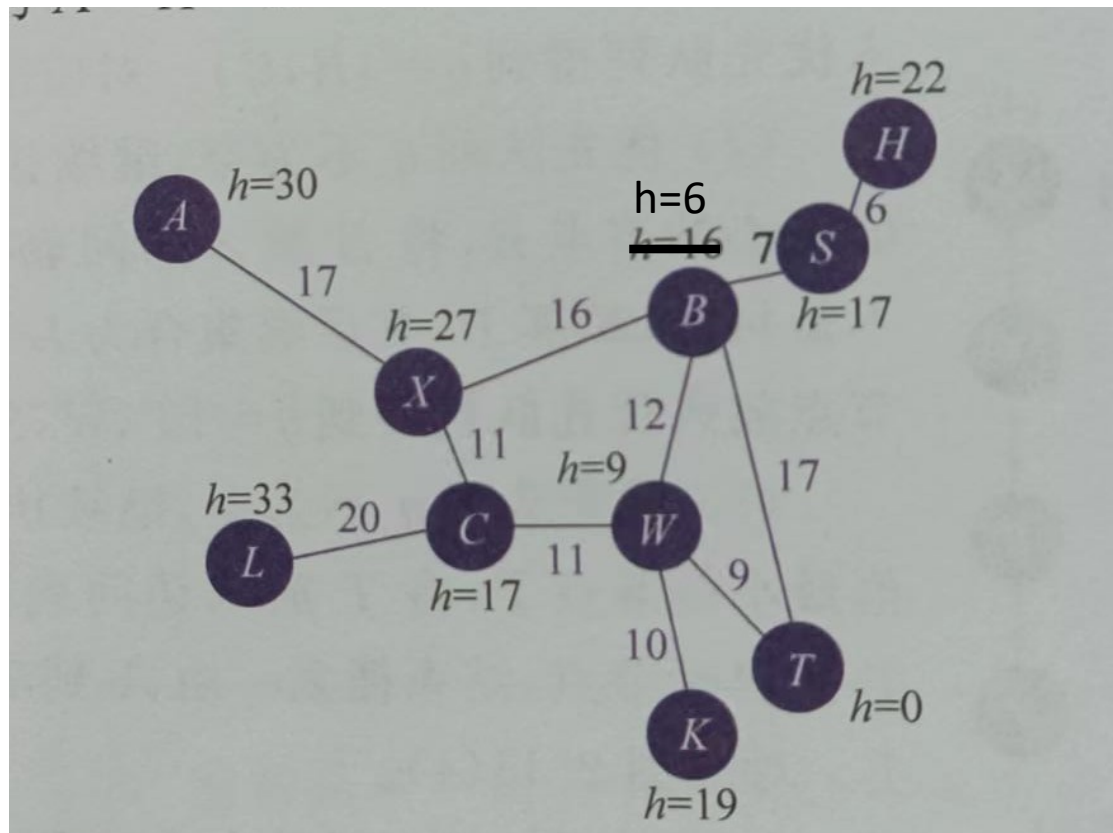


Homework A star 2: Running A* on the graph. A is start node, T is goal node. Here on node B, $h=16$ has been changed to $h=6$.

Can you get the same optimal path from A to T irrespective of whether $h=16$ or $h=6$ on node B? They are both admissible.

Give details as in **Homework A star 1**.

Is there some problem with the algorithm?



Homework: AStar3

Homework:

Problem 3.7 on page 114 of the textbook. Solve two questions.

- (a) How many states in this search problem?
- (b) Why shortest path goes along the edge of a polygon?

